

HANDS-ON 1: QA Concepts, Functional Testing & Defect Lifecycle Task 1: Map Testing Types to a Real System

1.1 Test Cases for Course Management API Unit Testing

text Test Case: Test the `validate_course_data()` function Description: Verify that the validation function correctly validates course data (name, code, credits, instructor) Type: Functional Isolation: Single function tested in isolation from the rest of the system Example: `def test_validate_course_data(): valid_data = {"name": "Data Structures", "code": "CS201", "credits": 4} assert validate_course_data(valid_data) == True`

```
invalid_data = {"name": "", "code": "CS201", "credits": 4}
assert validate_course_data(invalid_data) == False
```

Integration Testing

text Test Case: Test the `/api/courses/` POST endpoint with database integration Description: Verify that the API endpoint correctly saves course data to the database Type: Functional Components: API endpoint + Database + ORM Layer Example: Send POST request with valid course data, verify database has new record System Testing

text Test Case: Full end-to-end course creation flow Description: Test complete flow from UI form submission to database persistence and UI confirmation Type: Functional Scope: Entire system - Frontend + Backend + Database Example: Admin logs in → Navigates to course creation → Fills form → Submits → Verifies course appears in list User Acceptance Testing (UAT)

text Test Case: College admin creates a new course Description: Test from the actual end-user's perspective (college admin) Type: Functional User: College admin Example: Real admin user tests the complete workflow in staging environment 1.2 Functional vs Non-Functional Testing Functional Test Example

text Test: POST `/api/courses/` returns 201 status with correct course data Type: Functional (does it do what it should?) Verification: Status code, response body, database record Non-Functional Test Example

text Test: Response time for GET `/api/courses/` under 100 concurrent users Type: Non-Functional (how well does it do it?) Metric: Response time should be < 500ms for 95th percentile Tools: JMeter, LoadRunner, or Locust Other Non-Functional Tests:

text

1. Performance: How fast does the API respond?
 2. Security: Is the API protected against SQL injection and unauthorized access?
 3. Scalability: Can the API handle 1000 concurrent users?
 4. Reliability: Does the API work consistently without failures?
- 1.3 Black-Box vs White-Box Testing
- | Aspect | Black-Box Testing | White-Box Testing |
|--------------|---------------------------------------|------------------------------------|
| Definition | Testing without knowledge of internal | Testing with knowledge of internal |
| Focus | Input/Output behavior, functional | Code paths, logic, branches, loops |
| Performer | QA Tester | Developer / SDET |
| Requirements | requirements | |

```

-----+-----+-----+ | Techniques
| Equivalence partitioning, Boundary | Statement coverage, Branch coverage, || | value analysis | Path
coverage | +-----+-----+-----+
-+ | Example | Testing API without seeing the code | Testing individual functions with || | known
internal logic | +-----+-----+-----+
-----+ | When | System testing, UAT | Unit testing, Integration testing | +-----+-----+
-----+ | Advantage | Tests from user
perspective, | Can find hidden bugs, thorough || | independent of implementation | code coverage | +--
-----+ | Disadvantage | Limited coverage, may miss internal | Time-consuming, requires programming || | errors
| knowledge | +-----+-----+-----+
-----+ Real-World Example:

```

text Black-Box: A QA tester verifies that entering "CS201" in the course code field creates a course with that

code White-Box: A developer checks that the generate_course_id() function correctly increments the ID

counter and handles overflow 1.4 Formal Test Cases for POST /api/courses/ text +-----+-----+

```

-----+-----+-----+ | Test Case ID |
Description | Preconditions | Test Steps | Expected Result | Actual Result | Pass/Fail | +-----+-----+
-----+-----+-----+ | TC-API-001 |
Create course | API server | 1. Send POST | Status 201 |||| | with valid data | running, | request to | Created ||||
|| authentication | /api/courses/ | Course data |||| | token available | with valid | returned in |||| | JSON |
response |||| | 2. Check | Database record |||| | response | created |||| | status code |||| | 3. Verify
|||| | response |||| | body |||| | 4. Verify |||| | database ||| +-----+-----+
-+-----+-----+ | TC-API-002 | Create
course | API server | 1. Send POST | Status 400 |||| | with duplicate | running, | request with | Bad Request ||||
| course code | existing course | existing | Error: "Course |||| | with code | code "CS201" | code already ||||
"CS201" | 2. Check | exists" |||| | response | No duplicate |||| | status code | record created |||| | 3.
Verify |||| | error message ||| +-----+-----+
-----+-----+-----+ | TC-API-003 | Create course | API server | 1. Send POST | Status
400 |||| | with missing | running | request | Bad Request |||| | required field | without | Error: "Name is ||||
| "name" field | required" |||| | 2. Check | No record |||| | response | created |||| | status code |||| |
3. Verify |||| | validation |||| | error ||| +-----+-----+
-----+-----+-----+ Task 2: Defect Lifecycle & Severity Classification 2.1
Defect Lifecycle text +-----+-----+ | v | +-----+ | NEW | +-----+
+ || | v | +-----+ | ASSIGNED |-----> REJECTED | (Not a valid defect) | +-----+
-+ || | v | +-----+ | OPEN |-----> DEFERRED | (Fix postponed) | +-----+
v | +-----+ | FIXED |-----> Development fixes the defect | +-----+ | v | +-----+ | RETEST |----->
QA verifies the fix | +-----+ | v | +-----+ | VERIFIED |-----> REOPENED | (Fix failed) | +
---+-----+ | v | +-----+ | CLOSED |-----> Defect is resolved and closed | +-----+ | +
-----+

```

State Descriptions:

1. NEW: Defect is reported by QA
2. ASSIGNED: Defect is assigned to a developer
3. OPEN: Developer is working on the fix
4. FIXED: Developer has fixed the defect and pushed the code

5. RETEST: QA is testing the fix
 6. VERIFIED: QA confirms the fix works
 7. CLOSED: Defect is resolved and closed
 8. REJECTED: Not a valid defect (rejected by developer/PM)
 9. DEFERRED: Fix postponed to future release
 10. REOPENED: Fix didn't work, defect is reopened
- 2.2 Severity and Priority Classification Bug (a): API returns 500 error when creating a course

text Severity: Critical Priority: P1 Justification: Complete feature failure - course creation is a core business function. Users cannot add new courses, blocking a critical workflow. Bug (b): UI displays course name with wrong capitalization

text Severity: Low Priority: P3 Justification: Visual issue only, no functional impact. Users can still use the system correctly. Bug (c): Search returns results after 2 seconds (expected 500ms)

text Severity: Medium Priority: P2 Justification: Performance degradation affects user experience but doesn't break functionality. Users can still get results, just slower. Bug (d): Intermittent authentication failure (10% of attempts)

text Severity: High Priority: P1 Justification: Hard to reproduce (intermittent), affects critical login functionality. This type of bug indicates deeper instability and erodes user trust.

2.3 Complete Defect Report text

DEFECT REPORT

=====

DEFECT ID: DEF-2026-001 TITLE: Course Creation API Returns Internal Server Error (500) ENVIRONMENT: Staging (<https://staging-api.courseportal.com>) BUILD VERSION: v2.3.1 SEVERITY: Critical PRIORITY: P1 REPORTED BY: QA Team REPORTED DATE: 2026-07-20 ASSIGNED TO: Development Team STATUS: Assigned

STEPS TO REPRODUCE:

1. Navigate to Admin Panel -> Course Management
2. Click "Add New Course" button
3. Fill in the following fields:
 - Course Name: Data Structures
 - Course Code: CS201
 - Credits: 4
 - Instructor: Dr. Smith

4. Click "Save" or "Submit" button

EXPECTED RESULT:

- HTTP Status Code: 201 Created
- Course data returned in response body
- New course appears in the course list
- Course is saved to the database

ACTUAL RESULT:

- HTTP Status Code: 500 Internal Server Error
- Error Message: "Internal Server Error - Database connection failed"
- Course is NOT created
- Course does NOT appear in the list
- Database record NOT created

ATTACHMENTS:

- screenshot_of_500_error.png
- api_request_log.txt
- server_logs.txt

ADDITIONAL INFORMATION:

- Issue is 100% reproducible
- Occurs on all browsers (Chrome, Firefox, Edge)
- Affects all users with admin privileges
- First reported at: 2026-07-20 10:30 AM IST
- Last reproduced at: 2026-07-20 11:15 AM IST

WORKAROUND:

- No workaround available
- Courses cannot be created until fix is deployed
- Database backup can be restored if needed

ADDITIONAL NOTES:

- Stack trace indicates database connection pool exhaustion
- Suggests connection leaks in previous API calls
- Developer to investigate connection management

=====

===== 2.4 Severity vs Priority - Real-World Example Severity: Impact of the defect on the system functionality Priority: Urgency of fixing the defect based on business needs

Example 1: High Severity, Low Priority

text Defect: Year-end financial report calculates tax incorrectly (off by \$0.01) Severity: Critical - Incorrect financial calculations Priority: P4 - Year-end report is generated once per year Why?: Although technically severe (wrong calculations), it doesn't need immediate fix because the feature is rarely used and a workaround (manual calculation) exists Example 2: Low Severity, High Priority

text Defect: CEO dashboard displays company logo with a pixel distortion Severity: Low - Cosmetic issue, no functional impact Priority: P1 - CEO reviews dashboard daily Why?: Even though it's low severity, the CEO's daily review makes it high priority because it affects executive perception Key Takeaways text

1. Testing Levels: Unit -> Integration -> System -> UAT
2. Functional vs Non-Functional: Does it work vs How well does it work
3. Black-Box: QA tests without internal code knowledge
4. White-Box: Developers test with code knowledge
5. Defect Lifecycle: New -> Assigned -> Open -> Fixed -> Retest -> Verified -> Closed
6. Severity vs Priority: Impact on system vs Urgency of fix
7. Test Cases: Documented, repeatable procedures
8. Defect Reports: Complete information for developers to fix